

1 Robot Setup

1.1 Robot Model

The robot was implemented using a custom URDF/Xacro model and successfully spawned in Gazebo. The model includes the main base, wheel links, IMU link, camera link, and camera optical frame.

- Robot description file: [robot_description.xacro]
- File location: [submitted_folder/path/to/urdf/or/xacro]

1.2 TF Tree

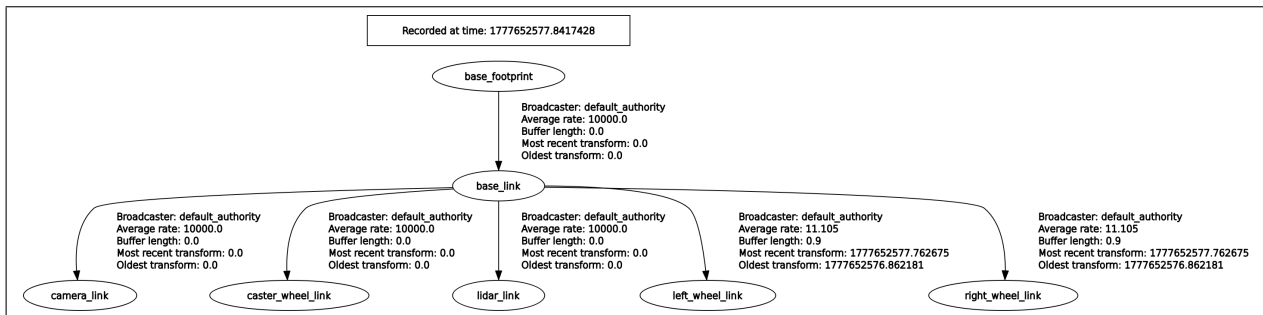


Figure 1: TF tree exported from `rqt_tf_tree`.

1.3 Frame Explanation

The robot follows the standard ROS coordinate convention: X forward, Y left, and Z upward.

- `base_footprint`: Ground projection of the robot base, used as the planar reference frame.
- `base_link`: Main body frame of the robot and parent frame for wheels and sensors.
- `left_wheel_link`: Left wheel frame connected to `base_link`.
- `right_wheel_link`: Right wheel frame connected to `base_link`.
- `imu_link`: IMU mounting frame fixed to the robot base.
- `camera_link`: Physical camera frame mounted at the front of the robot.
- `camera_optical_frame`: Optical camera frame, with Z forward, X right, and Y down.

2 Sensor Integration

2.1 IMU

The IMU was added using a Gazebo plugin and publishes data to:

```
/imu
```


3.2 Drift

The simulated IMU did not show significant long-term drift while the robot was stationary. Any small variations remained bounded and did not noticeably accumulate over time.

3.3 Simulation Fidelity

Compared to a real IMU, the Gazebo IMU is more ideal and less affected by temperature changes, mechanical vibration, calibration error, and long-term drift. It is useful for testing ROS 2 integration, but realistic noise and bias parameters are needed for a closer real-world comparison.

4 Environment Setup

4.1 Custom World

The Gazebo world file loads successfully and contains four ArUco marker poles arranged in a square pattern.

- World file: `poles_world.sdf`

4.2 Marker Coordinates

Marker Pole	X	Y
Marker 1	2	2
Marker 2	2	-2
Marker 3	-2	2
Marker 4	-2	-2

Table 1: ArUco marker pole coordinates.

4.3 World View

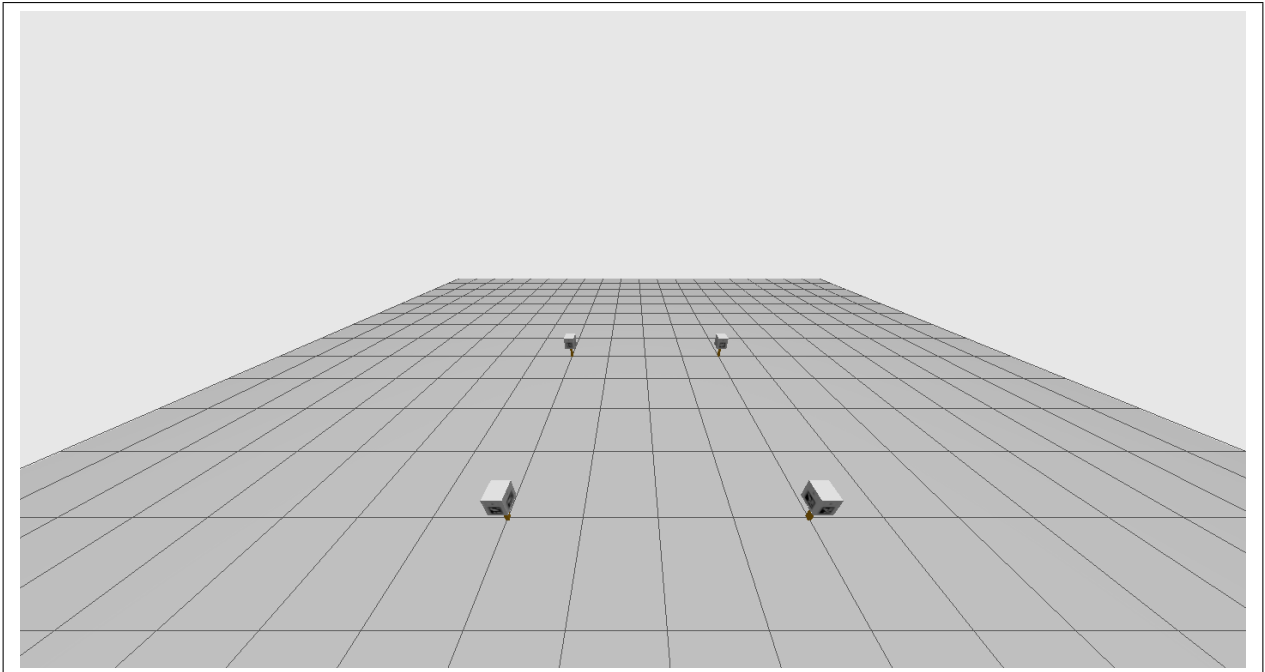


Figure 4: Gazebo world showing four ArUco marker poles arranged in a square pattern.

4.4 Startup Camera View

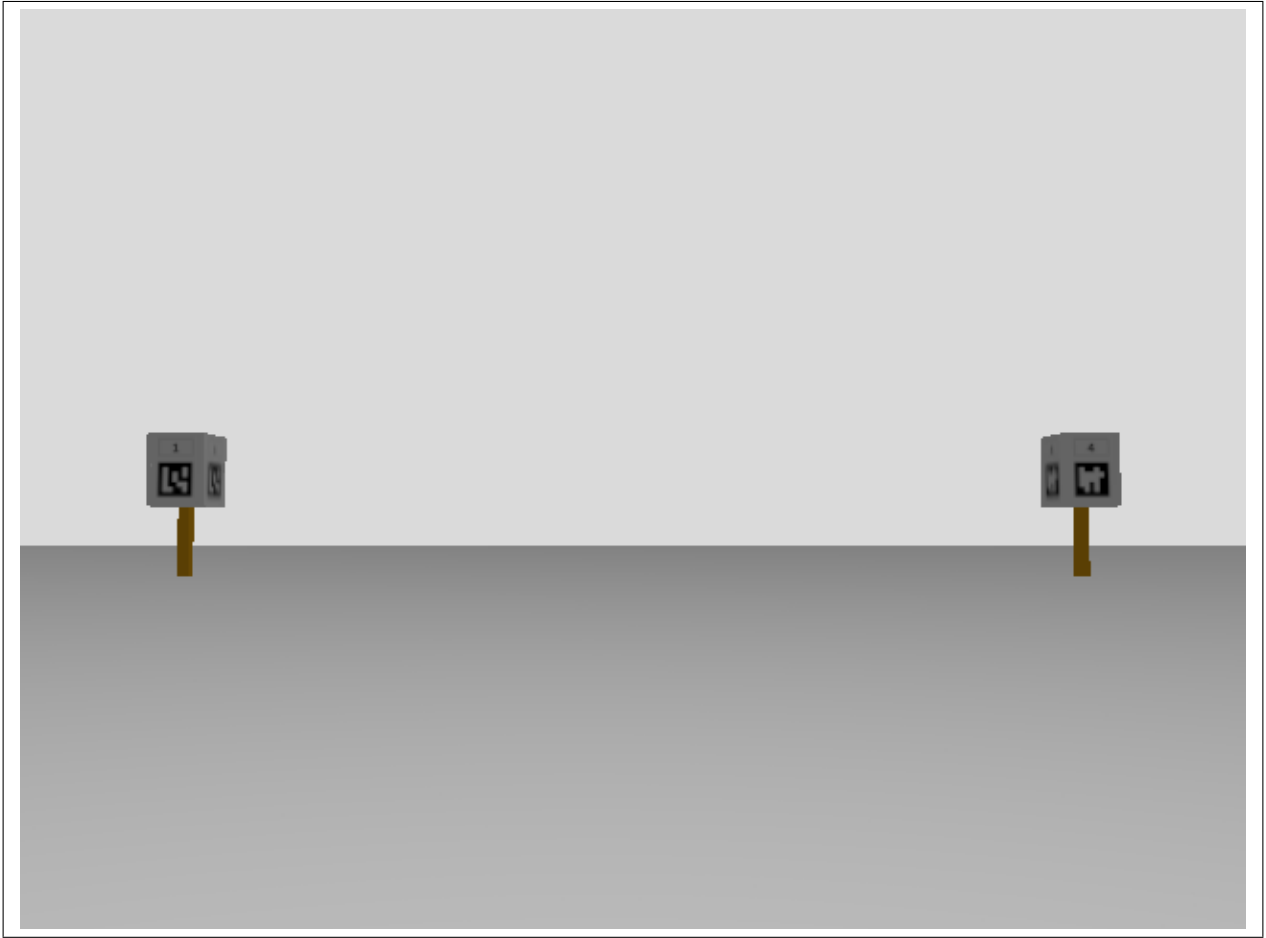


Figure 5: Robot camera view showing at least two visible ArUco markers at startup.

5 Perception Integration

5.1 Detection Node

The ArUco detection node was launched and connected to the robot camera topic. The node publishes detected marker pose data.

```
ros2 topic echo /perception/detected_aruco
```

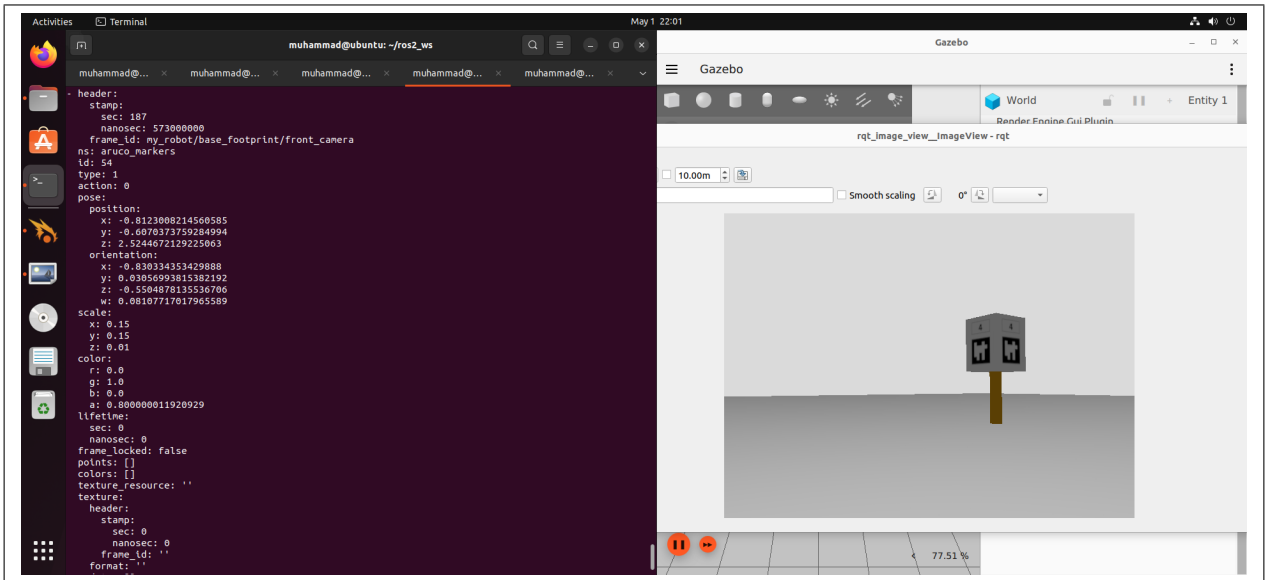


Figure 6: ArUco detection node running and publishing marker pose data.

5.2 Robot TF in RViz

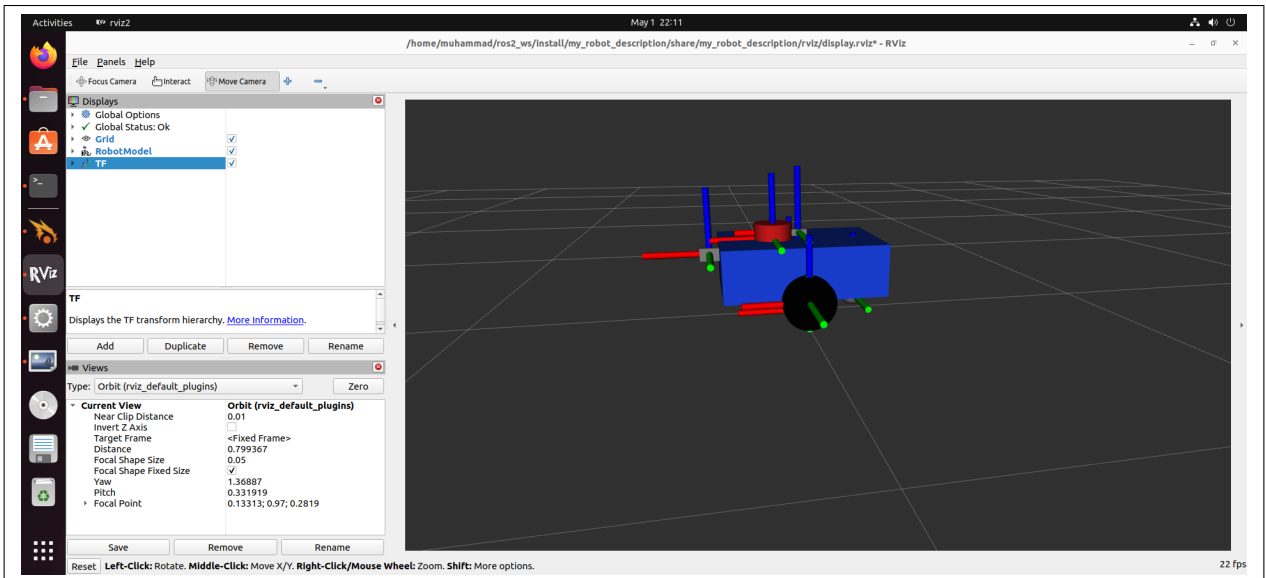


Figure 7: Robot TF frames visible in RViz.

5.3 Marker Frames in RViz

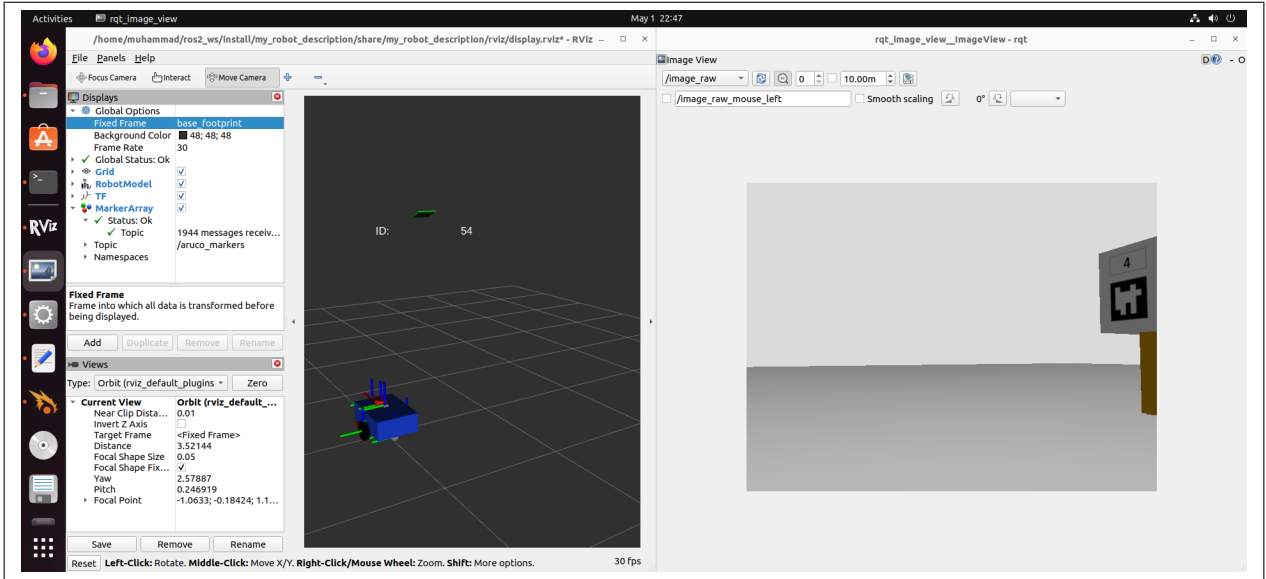


Figure 8: Detected ArUco marker frames overlaid correctly in RViz.

6 System Verification

6.1 Node Graph

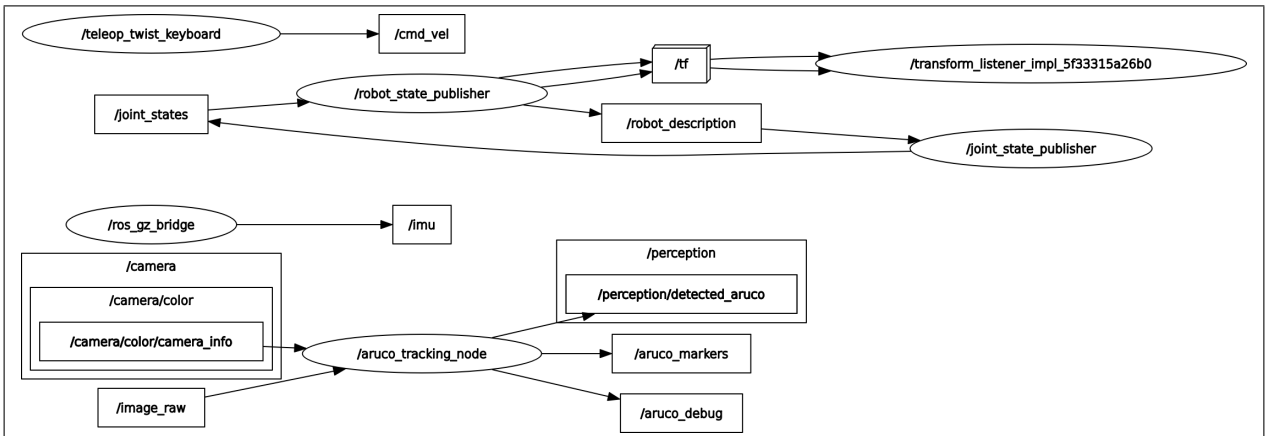


Figure 9: rqt_graph showing all nodes and topic connections.

6.2 TF Tree Export

The TF tree was exported using:

```
ros2 run tf2_tools view_frames
```

6.3 Sensor-to-Perception Flow

```
ros2 topic hz /image_raw
ros2 topic hz /imu
ros2 topic echo /perception/detection_aruco --once
```

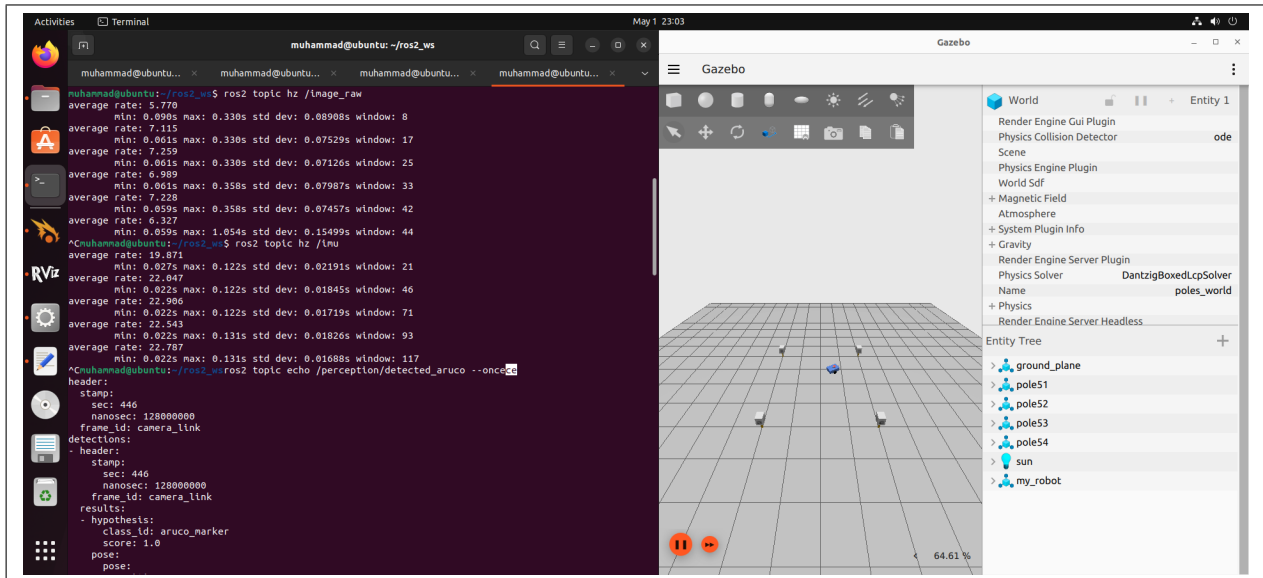


Figure 10: Evidence of sensor-to-perception data flow.